AI Contract Risk Analyzer

Product Requirements & Design Document

1. Product Strategy & Scope

Target Audience: Freelancers, agency owners, and small business operators who need a "first line of defense" before signing contracts, but cannot afford a lawyer for every standard agreement.

Core Value Proposition: Transform dense, intimidating legal jargon into a clear, actionable risk dashboard in under 60 seconds.

User Epics & Stories (MVP Scope Lock):

- Epic 1: Ingestion. As a user, I want to upload a PDF/DOCX so the system can read my contract.
- Epic 2: Analysis. As a user, I want the AI to highlight high-risk clauses (liability, termination, auto-renewal) so I know what to negotiate.
- Epic 3: Actionability. As a user, I want a quantifiable "Risk Score" and a downloadable PDF report to share with my team or client.

Constraint Check: We are intentionally omitting user authentication, historical database storage, and OCR for the MVP to ensure completion.

2. Technical Architecture (Solution Architect)

The architecture relies on a strictly linear, synchronous data flow. By using Streamlit, we merge the frontend and backend into a single Python application.

Tech Stack:

- Framework: streamlit (UI and state management)
- Parsing: PyMuPDF (PDFs), python-docx (Word docs)
- AI Integration: openai or other Ai SDK kit
- Reporting: reportlab or pdfkit

Data Flow:

1. 1. File Upload: User uploads via Streamlit.
2. 2. Text Extraction: Python scripts parse the file into a single raw text string.
3. 3. API Call: The text string is injected into a strict prompt template and sent to the LLM.
4. 4. JSON Response: The LLM returns a structured JSON object containing the score, clause list, and recommendations.
5. 5. Rendering: Streamlit parses the JSON and renders the dashboard.

3. AI & Prompt

Since coding is a secondary strength, we will offload the heavy lifting to the LLM using Structured Outputs (JSON Mode).

The Prompt Strategy:

You are an expert contract lawyer. Analyze the following contract text.
Identify key clauses related to: Liability, Termination, Payment, Auto-Renewal, and Confidentiality.

Respond STRICTLY in JSON format with the following structure:

```
{
 "overall_score": [0-100 integer, where lower is riskier],
 "overall_risk_level": ["LOW", "MEDIUM", "HIGH"],
 "clauses": [
  {
   "name": "Clause Name",
   "risk_level": ["LOW", "MEDIUM", "HIGH"],
   "explanation": "Why this is risky in 1 sentence.",
   "recommendation": "What to do about it."
  }
 ],
 "summary_recommendation": "Overall advice before signing."
}
```

Contract Text:
{contract_text}

4. User Experience & Interface (UX Designer)

Streamlit is block-based, so the layout must flow top-to-bottom logically.

Screen Layout Breakdown:

6. 1. Hero Section: Clean title and subtitle.
7. 2. Input Zone: A prominent drag-and-drop box.
8. 3. Processing State: A spinner with reassuring text.
9. 4. Results - Top-Level: A large metric displaying the overall score, color-coded.
10. 5. Results - Details: Use Streamlit's expander for each clause.

UI Element	Visual Representation
High Risk (Liability)	🔴 Red Expander Box. Highlights unlimited caps.
Medium Risk (Termination)	🟠 Amber Expander Box. Notes 90-day lock-ins.
Low Risk (Confidentiality)	🟢 Green Expander Box. Standard NDA

	terms.

5. QA

- The "Scanned Image" PDF: PyMuPDF extracts text, not images. Fix: Check if text < 50 chars, show error to upload text-based PDF.
- The Overly Long Contract: The LLM might timeout. Fix: Cap upload size to ~15 pages (10k words).
- Hallucinations: AI inventing clauses. Fix: Add strict system instruction to only analyze explicitly present clauses.

6. Execution & Delivery Plan

- The Skeleton (Foundation)

Goal: Get a Streamlit app running locally that accepts a file, extracts text, and prints it.

- The Brain (AI Integration)

Goal: Connect API key, send text using JSON prompt, print JSON.

- The Paint (Dashboard & UI)

Goal: Map JSON to metrics, columns, and expanders. Add PDF report button.

- The Showroom (Polish & Deploy)

Goal: Deploy on Streamlit Cloud, write README, record demo.